Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3676642.3736121

RESEARCH-ARTICLE

Voyager: Input-Adaptive Algebraic Transformations for High-Performance Graph Neural Networks

YANGJIE ZHOU, Shanghai Jiao Tong University, Shanghai, China

WENTING SHEN, Alibaba Group Holding Limited, Hangzhou, Zhejiang, China

JINGWEN LENG, Shanghai Jiao Tong University, Shanghai, China

SHUWEN LU, Shanghai Jiao Tong University, Shanghai, China

ZIHAN LIU, Shanghai Jiao Tong University, Shanghai, China

WEIHAO CUI, Shanghai Jiao Tong University, Shanghai, China

View all

Open Access Support provided by:

Shanghai Jiao Tong University

Alibaba Group Holding Limited

Peking University

China University of Geosciences

PDF Download
3676642.3736121.pdf
26 February 2026
Total Citations: 0
Total Downloads: 942

Published: 06 August 2025

Citation in BibTeX format

ASPLOS '25: 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems
March 30 - April 3, 2025
Rotterdam, Netherlands

Conference Sponsors:
SIGPLAN
SIGOPS
SIGARCH

Voyager: Input-Adaptive Algebraic Transformations for High-Performance Graph Neural Networks

Yangjie Zhou
yj_zhou@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China

Wenting Shen
wenting.swt@alibaba-inc.com
Alibaba Group
Hangzhou, China

Jingwen Leng*
leng-jw@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China

Shuwen Lu
lushuwen@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China

Zihan Liu
altair.liu@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China

Weihao Cui
weihao@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Zhendong Zhang
zhangzd@sqz.ac.cn
Shanghai Qi Zhi Institute
Shanghai, China

Wencong Xiao
wencong.xwc@alibaba-inc.com
Alibaba Group
Hangzhou, China

Baole Ai
baole.abl@alibaba-inc.com
Alibaba Group
Hangzhou, China

Yong Li
jiufeng.ly@alibaba-inc.com
Alibaba Group
Hangzhou, China

Wei Lin
weilin.lw@alibaba-inc.com
Alibaba Group
Hangzhou, China

Deze Zeng
deze@cug.edu.cn
China University of Geosciences
Wuhan, China

Yun Liang
ericlyun@pku.edu.cn
Peking University
Beijing, China

Quan Chen
chen-quan@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Ning Liu
ningliu@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Minyi Guo
guo-my@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China

Abstract

Graph neural networks (GNNs) are gaining popularity in diverse application domains and growing in complexity. As a result, it is crucial to achieve high-performance GNN execution. Among various techniques, algebraic transformations, including operator reordering and operator fusion, have been successfully applied to improve the computation and memory access efficiencies of DNN models. However, directly applying these methods to GNNs results in suboptimal performance. This is primarily due to the diversity of input graph structures, making it challenging to predict performance benefits. Additionally, the complex dependencies introduced by these input graph structures limit the effectiveness of existing optimization techniques.

To address these challenges, we propose *Voyager*, an input-adaptive compiler designed specifically for GNNs. Unlike existing methods that rely on static optimization rules, *Voyager* dynamically generates and evaluates a wide range of algebraic transformation candidates, including novel reordering and fusion strategies tailored to graph characteristics. By leveraging symbolic graph attributes, *Voyager* efficiently estimates tensor shapes and prunes low-performance candidates. A runtime selection module further enhances performance by choosing the optimal transformation based on real-time input features. Our evaluation shows that *Voyager*

*Jingwen Leng is the corresponding author of this paper.

This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '25, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1080-3/2025/03
https://doi.org/10.1145/3676642.3736121

achieves an average 3.5× speedup on GPUs over the state-of-the-art works on representative GNNs and datasets.

CCS Concepts: • **Computing methodologies → Parallel algorithms**; **Symbolic and algebraic algorithms**.

Keywords: Graph Neural Networks, Compilation Optimization, High-Performance Computing

ACM Reference Format:
Yangjie Zhou, Wenting Shen, Jingwen Leng, Shuwen Lu, Zihan Liu, Weihao Cui, Zhendong Zhang, Wencong Xiao, Baole Ai, Yong Li, Wei Lin, Deze Zeng, Yun Liang, Quan Chen, Ning Liu, and Minyi Guo. 2025. Voyager: Input-Adaptive Algebraic Transformations for High-Performance Graph Neural Networks. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3676642.3736121

1 Introduction

Graph neural networks (GNNs) represent significant advancements in deep learning, tailored for graph structures and features. By learning and exploring high-level representations of graphs, GNNs have significantly advanced fields like chemistry [14], neurology [1], anomaly detection [9, 33], social networks [44] and recommendations [10, 41]. However, as GNN models increase in complexity, efficiently processing and executing them on advanced parallel hardware like GPUs becomes more challenging.

Algebraic transformation has emerged as a crucial strategy for enhancing the computational efficiency of deep neural networks [25, 45, 58, 67, 74]. Its basic idea is to leverage computation equivalence to execute the underlying model more efficiently. Two commonly used techniques are operator reordering, which reduces computational overhead by exploiting the associative and commutative properties of operators, and operator fusion, which merges multiple operators to reduce memory traffic and kernel launch overhead. However, applying these techniques to GNNs introduces unique challenges. Unlike DNNs, GNNs traverse and compute along graph structures, where irregular memory access patterns and complex data dependencies between vertices and edges hinder the direct application of traditional reordering and fusion techniques. This necessitates a more specialized approach to optimize GNN execution.

In this work, we show that recent studies on algebraic transformations for GNNs [63, 65] suffer from two major limitations: a restricted optimization space and the inability to adapt to diverse input graphs. For instance, existing GNN reordering methods primarily apply fixed rules during offline compilation. Although this strategy offers a certain level of optimization with predictable gains, it exhibits inadequacies for the growing complexity of GNN models. Illustratively, our findings reveal that by deploying iterative combinations of multiple rules for operator reordering, we can achieve up to 9.1× final speedup for the first layer of the Graph Isomorphism Network (GIN) model [66] (Sec. 2.2). Existing reordering methodologies, unfortunately, fail to capitalize on such substantial optimization opportunities, emphasizing the necessity for more sophisticated reordering strategies.

Moreover, we find that current operator fusion transformations for GNNs are similarly limited by their reliance on fixed strategies. The majority of operators in GNNs are intricately linked to graph traversal, which plays a key role in determining the efficiency of operator fusion. All existing GNN frameworks [12, 60] and compilers [63, 65] implement operator fusion via pull-based graph traversal, where the destination vertex serves as the central vertex for traversing its outgoing edges. However, our analysis reveals that a shift from pull to push-based graph traversal, where the source vertex becomes the centric vertex for traversing incoming edges [56, 71], can yield significant benefits by achieving more balanced computation and enabling more fusion opportunities in specific scenarios. By alternating the traversal direction, we show that a maximum speedup value of 5.3× can be achieved (Sec. 2.2). These findings suggest that augmenting the fusion strategy space through a flexible traversal-centric approach could further enhance the performance of GNNs.

To fully leverage these untapped optimization opportunities, we propose *Voyager*, an automatic, end-to-end compiler designed to enable comprehensive and input-aware algebraic transformations for GNNs. *Voyager* focuses on two key insights to facilitate the construction of a more expansive algebraic transformation strategy space.

First, *Voyager* utilizes graph-related attributes to efficiently build and prune the reordering strategy space. *Voyager* creates a graph-aware computational graph representation with graph-related attributes and propagates these attributes. We further establish a set of joint reorder rules catering to both basic graph and neural network (NN) operators. Integrating these joint rules with the strategy generation algorithm allows *Voyager* to construct an exhaustive reordering strategy space teeming with multiple candidates. Further, to achieve efficient candidate pruning, we employ an attribute-aware cost model, which prunes reordered candidates and retains as few of the most computationally efficient candidates as possible for subsequent optimization.

Second, *Voyager* focuses on graph traversal to analyze and construct a comprehensive strategy space for operator fusion. To maximize the exploration of fusion strategies, *Voyager* first extends the traditional pull-based fusion to include push-based fusion, where the traversal direction is reversed. By analyzing the read/write characteristics of existing fusion patterns, we introduce a hybrid fusion method that combines pull- and push-based fusion, offering a more flexible and comprehensive strategy space for operator fusion.

These procedures result in multiple optimized candidates tailored to the target GNN model. Unlike existing methods that employ static and input-agnostic strategies, *Voyager*

Table 1. Comparison of existing GNN algebraic transformation methods with our proposed *Voyager*.

Existing Work	Operator Reorder		Operator Fusion			Input-Adaptive
	Short-Range	Long-Range	Pull	Push	Hybrid	
Graphiler [65]	✓		✓			
Seastar [63]			✓			
Voyager(Ours)	✓	✓	✓	✓	✓	✓

considers input information as a critical factor in runtime strategy selection, ensuring optimal performance across different graph structures. Tbl. 1 compare existing GNN algebraic transformation method with *Voyager*, which illustrates how *Voyager* offers a broader strategy space, while leveraging an input-adaptive strategy selection approach.

We conduct comprehensive experiments on the enterprise V100 GPU and A100 GPU to evaluate *Voyager*'s performance against different state-of-the-art baselines, including GNN frameworks [12], compilers [63, 65], and other manual optimization [62] on five mainstream GNN models. The results show that *Voyager* achieves an average speedup of 3.5× compared to the baselines with negligible runtime overhead, demonstrating the effectiveness of *Voyager*.

Overall, our work makes the following contributions:

- We motivate the necessity of more sophisticated algebraic transformations for GNNs. Specifically, we identify the limitations of existing works, highlighting their untapped optimization space and input-aware opportunities.
- For operator reordering, we propose leveraging graph attributes and graph-NN joint reordering rules to address dynamic tensor shapes and streamline optimization space construction and pruning.
- For operator fusion, we introduce traversal-aware fusion that expands strategy space with push-based and hybrid methods, which significantly improves upon existing works that rely solely on pull-based fusion techniques.
- We propose *Voyager*, an end-to-end GNN compiler. By leveraging a graph-aware strategy space and input-adaptive techniques, *Voyager* consistently outperforms existing frameworks, compilers, and manual optimizations.

2 Background and Motivation

In this section, we first present the necessary GNN background and characterize the input diversity observed in real-world graphs. We further explore how algebraic transformations can enhance GNN performance under this diversity.

Table 2. Classification of operators supported in this work.

Category	Type	Description	Representative
Graph	`scatter`	Move data from vertex to edge	`scatter.in`, `scatter.out`
	`gather`	Collect and reduce data from edge to vertex	`gather.in.sum`, `gather.out.mean`
NN	`elw`	element-wise operator	`add`, `relu`
	`reduction`	Reduction-based operator	`matmul`, `conv`

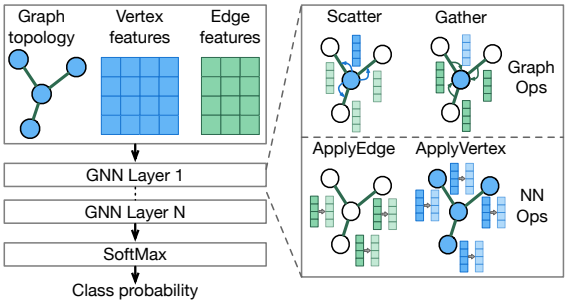


Figure 1. The general computation flow for GNN models.

2.1 GNN Computation and Input Diversity

GNN Computation Flow and Operations. GNNs have recently gained substantial attention from both academia and industry for their powerful learning capabilities on non-Euclidean graph structures [4, 64]. The general computational flow of GNNs is depicted in Fig. 1 [40]. The GNN's input comprises the input graph's topology structure and its associated dense feature embedding. Each GNN layer includes two primary types of operators to perform the computation: (1) graph operators (`GOPs`) and (2) neural network operators (`NN-OPs`). The operator classification, including representative examples, is further detailed in Tbl. 2.

We describe `GOPs` as operators that traverse the input graph structure. Two such fundamental operators are `scatter` and `gather`, which resemble vectorized graph propagation in conventional graph processing paradigms such as GAS [38]. The `scatter` operator distributes the embedding of each vertex to its outgoing (or incoming) edges, referred to as `scatter.out` (or `scatter.in`) in Tbl. 2. Conversely, the `gather` operator collects and reduces the embeddings of all the incoming (outgoing) edges of each vertex, denoted as `gather.in` (`gather.out`), yielding an updated vertex embedding. The `scatter` operator has only a copy function, while `gather` operator has different functions like sum/max/mean, referred to as `gather.sum/.max/.mean`.

We describe `NN-OPs` as operators that exist in DNNs and compute on dense tensors. They can be further categorized into two types based on their computational characteristics. The first, referred to as `elw`, applies a specific operation to each tensor element, leaving the input shape unaltered. The second `reduction` operator transforms a set of values into

Table 3. Details of graph datasets used for evaluation.

Dataset	#V	#E	#Feat.	#Cls	MaxDst	MaxSrc	StdDst	StdSrc
cora (CO)	2K	5K	1433	7	166	5	5.22	1.48
soc-BlogCatalog (SB)	88K	2.09M	128	39	9429	2538	206.81	57.98
ogbn-arxiv (OA)	169K	1.17M	128	40	13155	436	67.60	8.88
amazon0601 (AM06)	403K	3.39M	96	22	2751	10	15.28	2.79
amazon0505 (AM05)	410K	3.36M	96	22	2751	10	15.31	3.07
snap-patents (SP)	2.92M	13.97M	269	5	1679	5675	7.41	8.37
PROTEINS_full (PR)	43K	162K	29	2	25	25	1.15	1.15
TWITTER-Partial (TW)	580K	1.44M	1323	2	12	12	1.52	1.52
Yeast (YE)	1.71M	3.64M	74	2	6	6	0.75	0.75
SW-620H (SW)	1.89M	3.94M	66	2	5	5	1.16	1.16
Reddit (RE)	233K	114M	602	41	21657	21657	799.82	799.82
ogbn_products (OP)	2.44M	123M	200	107	17481	17481	95.90	95.90
com-orkut (OR)	3.07M	117M	128	2	3415	33007	51.09	130.97
enwiki-2013 (EW)	4.21M	101M	300	12	431795	8104	412.90	47.85

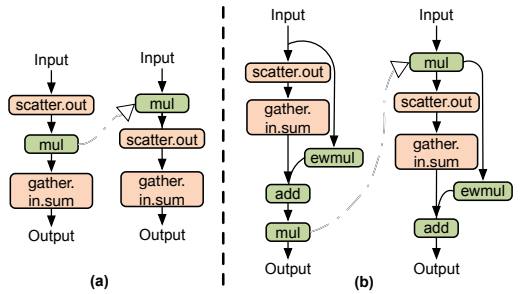


Figure 2. Short-range (a) and long-range (b) reordering.

a single value in certain dimensions, such as matrix-matrix multiplication (`matmul`) and convolution (`conv`).

Input Diversity of GNN. GNNs feature dynamic inputs that distinguish them from traditional DNNs and complicate optimization. To evaluate and understand GNN behavior, we analyze 14 commonly used graph datasets [12, 27, 32, 55], as presented in Tbl. 3. This dataset collection presents information on the quantity of vertices (#V) and edges (#E) in each graph, signifying their respective scales. These diversified graph datasets also demonstrate variations in features (#Feat.) and class sizes (#Cls). Additionally, we calculate the maximum degree of destination (MaxDst) and source vertices (MaxSrc), as well as the standard deviation of the degree, providing indicators of graph balance for both destination-major and source-major topologies (StdDst/StdSrc). For directed graphs, the imbalance between destination-major and source-major topologies can be significant.

The diversity of inputs significantly impacts both computation and memory access patterns. While prior work has leveraged this property in conventional graph processing [42, 68] and GNN operator-level· optimizations [75], we highlight its critical role in algebraic transformations for GNNs, which we elaborate on in the following subsections.

2.2 Reordering and Fusion Opportunities for GNNs

Long-Range Reordering Opportunity. Current GNN optimization research suggests reordering adjacent operators to attain deterministic benefits [65], a strategy we term *short-range reorder*. Fig. 2(a) exemplifies the short-range reordering between `scatter` and `matmul` operators within existing GNN compiler [65]. This reordering strategy eliminates the redundant `matmul` operations on edge embedding vectors, thus bringing *deterministic* benefits.

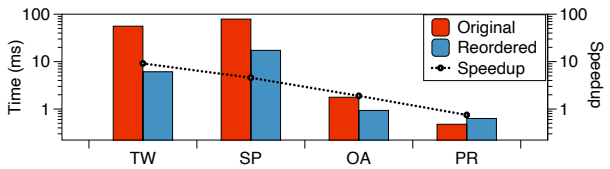


Figure 3. Performance comparison between the original and reordered GIN's first layer across four datasets on V100 GPU.

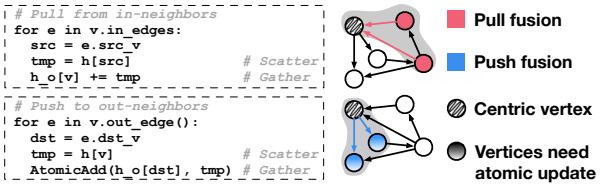


Figure 4. Illustration of existing pull fusion, and proposed push fusion for the fused-aggregation operator.

However, as GNN models grow in complexity, short-range reordering becomes insufficient to capture all optimization opportunities. As demonstrated in Fig. 2(b), in more complex models like GIN [66], there are extensive dependencies between operations that span across the computation graph. To address this, *long-range reordering* becomes necessary, where operators are reordered across larger sections of the graph to capture these dependencies.

To validate the impact of long-range reordering, we manually implemented this approach for the GIN model shown in Fig. 2(b) using DGL [60], while ensuring computational equivalence. The results in Fig. 3 show that datasets such as `TW` experience nearly 10× performance improvement, while others, like `PR`, exhibit negative results. This performance variability is highly influenced by the differences in feature size and graph structure, revealing that the effectiveness of reordering strategies is highly *input-dependent*.

Exploiting long-range reordering presents two key challenges. First, it requires multiple short-range reordering steps, during which intermediate steps may temporarily degrade performance. Traditional methods often prematurely halt these transformations, leading to suboptimal performance. Second, the optimal reordering strategy varies across input graphs, as no single reordering is universally effective.

Traversal-Aware Fusion Opportunity. Within existing GNN frameworks and compilers, operator fusion serves as a pivotal component, typically implemented through pull-based graph traversal and associated fusion templates. However, current GNN compilers overlook the potential of push-based traversal, a strategy extensively studied in traditional graph processing [56, 71] and hardware acceleration [13]. Inspired by these prior techniques, we show that push-based traversal offers two key advantages in GNNs: more balanced computation on specific directed input graphs, and broader fusion scopes for both undirected and directed graphs.

To illustrate this optimization, we use the example in Fig. 4 from GCN model [30]. For simplicity, we omit the feature dimension loop in the illustration. Current framework [60] and compiler [65] fuse `scatter.out` and `gather.in.sum` operators into a `fused-aggregation` operator using pull-based traversal, depicted in the upper half of Fig. 4. In this strategy, each centric vertex is responsible for gathering embeddings from its incoming edges. The fused operator combines two logical operations. The first operation, `scatter.out`,

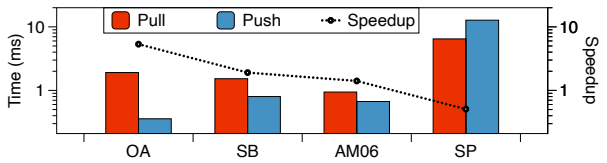


Figure 5. Performance comparison of pull- and push-based fused-aggregation across various datasets on V100 GPU.

extracts the embedding from each source vertex and temporarily holds it. The second operation, `gather.in.sum`, accumulates this embedding into the centric vertex's output buffer. This fused operator allows each vertex to pull and aggregate all its neighbors within a single work entity, such as one CUDA thread.

The lower part of Fig. 4 shows the push-based fused aggregation, where each centric vertex actively pushes its own embedding to all outgoing neighbors. This strategy applies the same fused scatter and gather operations as the pull-based strategy, achieving equivalent aggregation semantics but reverses the traversal direction by iterating over outgoing edges. Since multiple centric vertices may simultaneously push embeddings to the same destination vertex, atomic updates are required to ensure correctness.

While both pull- and push-based fusion strategies introduce computational imbalances, the degree of imbalance differs depending on the traversal strategy on the same graph. Specifically, pull-based strategies tend to create imbalances due to the variation in incoming edges, whereas push-based strategies introduce imbalances due to differing outgoing edge counts. Consequently, different traversal methods yield varying degrees of computational balance for a given graph.

We evaluated the performance of these two different fusion strategies for the `fused-aggregation` operator on V100 GPU with different datasets. As depicted in Fig. 5, different datasets demonstrate distinct preferences for the two strategies. For instance, datasets `OA`, `SB` and `AM06` have much better performance with push-based fusion, while dataset `SP` prefers pull-based fusion. This variation in performance can be attributed to differences in dataset topology. For instance, the `OA` dataset has a more balanced degree of outgoing edges than incoming edges (StdSrc = 8.88 vs. StdDst = 67.6, as shown in Tbl. 3), making push-based fusion more effective despite the overhead of atomic updates.

Another key advantage of push-based traversal is its potential to extend the fusion chain for GNNs. Pull-based traversal is limited to fusing a single `gather` operator due to the strict producer-consumer relationship. In contrast, combining pull- and push-based traversal can overcome this limitation by exploiting more fine-grained producer-consumer relationships. Further details on this aspect are provided in Sec. 5.

Summary. We have shown that algebraic transformation optimization is crucial for GNNs as they grow more complex and encounter dynamic inputs. These factors introduce challenges such as deeper operator dependencies and unpredictable performance variations. Our findings emphasize the need for a comprehensive strategy space for algebraic transformations, combined with adaptive selection mechanisms, to effectively manage the variability in models and inputs.

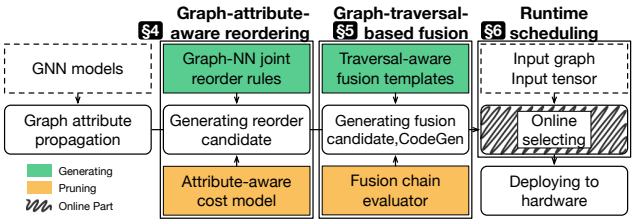


Figure 6. Design overview of *Voyager*.

3 Overview of *Voyager*

This section provides an overview of *Voyager*, an end-to-end compiler designed to optimize algebraic transformations for GNNs. As shown in Fig. 6, *Voyager* consists of three key modules: graph-attribute-aware reordering, graph-traversal-based fusion, and runtime scheduling. These modules work collaboratively to facilitate algebraic transformations for GNNs by constructing a broad strategy space, removing suboptimal candidates, and tailoring strategies to the specific input graph's characteristics.

In the offline phase, *Voyager* first constructs the computational graph for a given GNN model and extends it with graph-related attributes. These attributes are propagated throughout the computational graph, forming the basis for the graph-attribute-aware reordering module. This module applies joint reordering rules across both graph and neural network operators, leveraging long-range reordering opportunities to streamline operations. This module generates multiple reorder candidates to fully exploit long-range reordering opportunities and prunes low-performance candidates using an attribute-aware cost model, which handles the challenges of dynamic tensor shapes in GNNs.

The second module, graph-traversal-based fusion, explores a wide range of fusion strategies by focusing on the traversal direction of graph operators. Unlike traditional methods that rely solely on pull-based fusion, *Voyager* integrates push-based and hybrid fusion techniques. This extension broadens the fusion strategy space and enables more efficient memory access patterns. A fusion chain evaluator ensures that only legal candidates are selected for the next phase.

Finally, the runtime scheduling module adapts the selected reordering and fusion strategies based on the real-time characteristics of the input graph and tensors. This adaptive scheduling minimizes overhead by dynamically selecting the most efficient strategy for each graph structure, ensuring high performance across varying input graphs.

In summary, *Voyager* offers a robust, end-to-end solution for optimizing algebraic transformations in GNNs. By incorporating graph-centric and input-adaptive techniques,

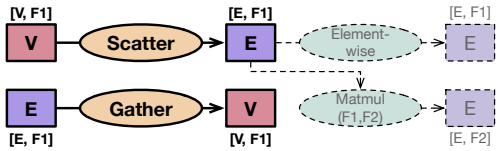


Figure 7. Graph attribute propagation examples. 'V' ('E') denotes vertex (edge) tensor. `GOPs` modify tensor attributes, while `NN-OPs` do not.

Voyager effectively generates high-performance code for GNNs with diverse models and input graphs.

4 Graph-attribute-aware Reordering

In this section, we introduce the operator reordering module in *Voyager*. We start by enhancing the conventional computational graph with graph-related properties to broaden its functionalities. This graph-centric approach enables the creation of a unified strategy space, facilitating comprehensive operator reordering optimization and effective pruning.

4.1 Graph-Related Attributes Propagation

The challenge of operator reordering for GNNs lies in the input-graph-dependent tensor shapes, hindering the decision on whether reordering would reduce computation costs (Sec. 2.2). To address this challenge, we propose leveraging the graph-related attributes in the computational graph, which allows us to compare the computation costs before and after applying a reordering rule.

We define the input-graph-level semantics for each tensor in a GNN model, depending on its association with vertex or edge. Tensors associated with vertices are called vertex tensors, while those linked to edges are termed edge tensors. We establish rules for graph-related attribute propagation using fundamental `GOPs`, specifically `scatter` and `gather` operators, as illustrated in Fig. 7. The `scatter` operator transforms a vertex tensor into an edge tensor, while the `gather` operator performs the converse. To summarize, these `GOPs` modify the attributes of the tensor tied to the input-graph dimension. The remaining `NN-OPs` leave the tensor type untouched during attribute propagation. Additionally, the `elw` operator maintains the attributes of the feature dimension, while `reduction` operator such as `matmul` could potentially modify the feature parameter.

By inferring the graph-related tensor type, we can further infer the graph semantics of the tensor shape and use it to estimate the computational cost. The key is to use graph-related special placeholders to mark unknown shape parameters. Specifically, we mark the first dimensional shape of a vertex tensor as 'V', and that of an edge tensor as 'E'. The remaining dimensions are marked as 'F'. These annotations play a crucial role in subsequent cost estimation.

Table 4. Joint reordering rules for Graph-NN operators.

No.	Computation graph structure	Computation graph structure in equation	Comment
1	$gather.sum(matmul(E.f, W))$	$matmul(gather.sum(E.f), W)$	commutativity
2	$gather.mean(matmul(E.f, W))$	$matmul(gather.mean(E.f), W)$	commutativity
3	$matmul(scatter(V.f), W)$	$scatter(matmul(V.f, W))$	commutativity
4	$elw(scatter(V.f))$	$scatter(elw(V.f))$	commutativity
5	$matmul(concat(A, B), W)$	$W_L, W_R = split(W)$ $ewadd(matmul(A, W_L), matmul(B, W_R))$	matmul def.
6	$matmul(ewadd(A, B), W)$	$ewadd(matmul(A, W), matmul(B, W))$	distributivity
7	$matmul(ewsub(A, B), W)$	$ewsub(matmul(A, W), matmul(B, W))$	distributivity
8	$matmul(ewmul(A, a), W)$	$ewmul(matmul(A, B), a)$	distributivity

Algorithm 1: Reordering candidates generation and pruning.

Input: Original graph-centric GNN Model $\mathcal{G}_{in}$, Reordering rules $\mathcal{P}$
Output: Reordered Candidates $\mathcal{S}$

```
// step 1: generate potential reordered candidates
D = newSet(); H = newQueue(); H.enque(G_in);
while H ≠ {} do
    G = H.dequeue();
    for each p in P do
        G' ← Reorder(G, p);
        if G' is valid then
            G' ← Optimize(G');
            if G' ∉ D then
                H.enqueue(G'); D ← D ∪ G';
// step 2: pruning reordered candidates
S = D; for each d1 in D do
    for each d2 in D do
        if comparableFlops(d1, d2) then
            S = S - maxFlops(d1, d2);
return S
```

4.2 Graph-NN Joint Reordering Rules

We now present the details of the reordering rules. Those rules listed in Tbl. 4 focus on unfused primitive operators, aiming to minimize the number of rules required. The first four rules in Tbl. 4 constitute the newly introduced reordering rules. These four rules are deemed valid due to the commutative property (last column in the table) exhibited by `gather.sum` and `gather.mean` operators, as well as the inherent nature of `scatter` operator to perform a copy operation, thereby enabling its reordering with matrix multiplication and element-wise operations. We show later that these rules are sufficient for GNNs studies in our work. Additionally, the remaining four NN-related reordering rules are directly derived from previous DNN compilers [25, 45].

4.3 Generating Reordering Candidates

With graph-related attributes, we design a two-step algorithm in Algo. 1 that automatically applies reordering rules and generates potential candidates for a GNN model.

The first step aims to generate all the valid candidates by iteratively applying our reordering rules listed in Tbl. 4. To achieve this, we design an algorithm that performs a breadth-first search (BFS). At first, the initial model is enqueued in $\mathcal{H}$. For each dequeued element from $\mathcal{H}$, we iteratively search its

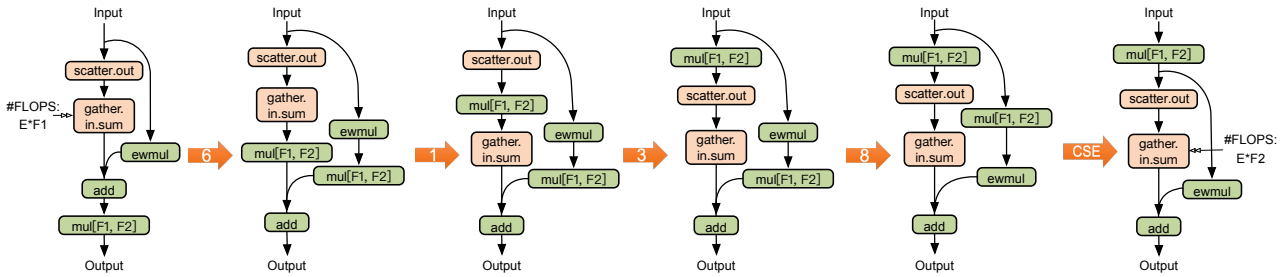


Figure 8. Long-range operator reordering for GIN. V (E) denotes the vertex (edge) count, and F1 (F2) denotes the input (output) feature size. We estimate the computation cost #FLOPS for each operator using these attributes. Each arrow indicates the reordering rule from Tbl. 4.

applicable reordering pattern. A successful match leads to a newly generated candidate, which is then optimized with typical methods including *dead operator elimination* [35] and *common subexpression elimination* [43]. The optimized candidate is added to the set $\mathcal{D}$ and the queue $\mathcal{H}$. This step repeats until $\mathcal{H}$ is empty.

In the second step, we construct a graph-attribute-based cost model to estimate the computation cost of a reorder candidate and prune candidates with high costs. Since the vertex count V and edge count E in an input graph are unknown at the compilation time, we propose a symbolic comparison approach based on graph attributes. We use the commonly understood assumption of real-world input graphs that V is much smaller than E [52], and leverage the number of floating point operations #FLOPS as the primary metric in the computation cost comparison. We adopt #FLOPS as a symbolic and generalizable proxy that effectively reflects the relative computational complexity, enabling efficient offline filtering without profiling. In this way, we prune as many candidates as possible via pair-wise comparison and retain those not comparable for the subsequent stages.

We use the subgraph from GIN model [66] in Fig. 8 as an example to illustrate these processes. In this example, we start from the leftmost original model, which matches the 6th rule in Tbl. 4 that matches, resulting in the second reordered candidate. We then further apply the first rule to generate the third candidate. We repeat this process until we have found all five candidates. After generating all valid candidates, we compare their computational costs. The second, third, and fourth candidates all have additional computational costs from the `matmul` operator, so we only retain the first and last candidates. With this strategy, the computation cost of the `gather` operator changes from the first version of $E \times F1$ to the last version of $E \times F2$, where $F1$ and $F2$ are runtime-determined feature dimension sizes. In this manner, *Voyager* exhibits the capability to comprehensively explore the strategy space for operator reordering optimization, even in scenarios where complete input information is lacking.

5 Graph-Traversal-Based Fusion

This section outlines the design of the graph-traversal-based fusion module. We first introduce novel fusion templates based on various traversal strategies, followed by identifying fusion candidates and generating efficient GPU kernels.

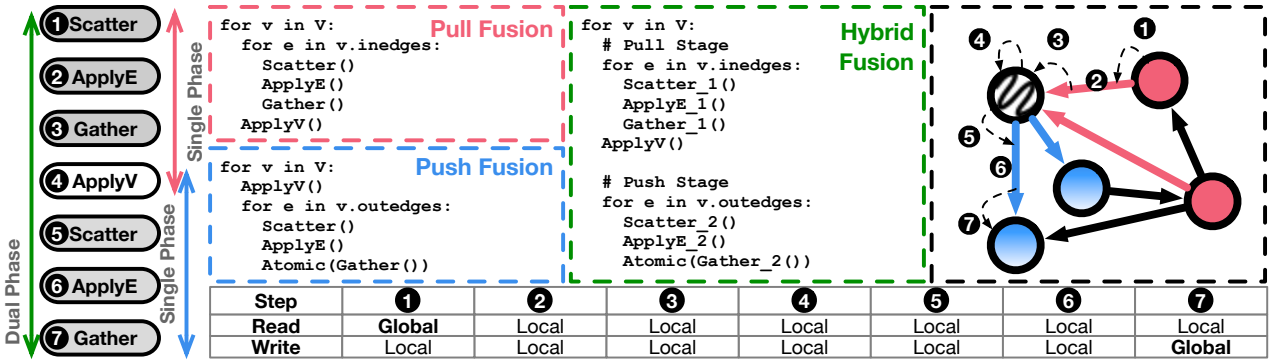
5.1 Traversal-Aware Fusion Templates

Our work takes a traversal-aware approach to explore the fusion strategy space. We follow the prior work and define a basic fusion unit as a vertex-to-vertex data communication along the input graph topology [40], which is commonly viewed as a complete execution phase in GNNs. We leverage vertex-level parallelism used in frameworks like DGL [60]. Under this paradigm, we deploy a fixed number of software-level workers (e.g., CTAs/Wraps on GPUs) within vertex-centric loops to parallelize computation. This setup allows each worker to perform identical operations on distinct data sets across multiple operations. Consequently, we exclude feature dimension analysis to enhance focus and conciseness.

Single Phase Fusion. We first consider the pull-based fusion template, which is the only choice in existing generic frameworks [12, 60], systems [40], and compilers [63, 65] for GNNs. As depicted in the top left of Fig. 9, this template requires each destination vertex to traverse its incoming edges. Echoing the `fused-aggregation` example in Fig. 4, this fusion template includes outer loops and inner computations. Within this template, the `scatter` operator triggers read accesses for each centric vertex to the *global* address space in the GPU memory hierarchy [48], while other operators both read and write the *local* address space.

While push-based traversal has been utilized in traditional graph processing systems [56, 71], we are the first to study it in the context of GNN optimizations. The left bottom of Fig. 9 illustrates an example of push-based traversal where each source vertex traverses its outgoing edges. In this push-based fusion template, multiple source vertices may update the same destination vertex, necessitating the use of `Atomic-gather` operator to avoid data conflicts. Contrasting the pull-based fusion, the `gather` operator in the push-based fusion writes to the global address space, while other operators read and write the local address space.

Dual Phase Fusion. Only using the pull or push-based traversal limits the fusion chain to a single phase. As previously analyzed, pull-based fusion template starts with global



Step	1	2	3	4	5	6	7
Read	Global	Local	Local	Local	Local	Local	Local
Write	Local	Local	Local	Local	Local	Local	Global

Figure 9. Illustration of pull-&-push-based fusion (single phase) and hybrid-based fusion (dual phase), with corresponding memory access behaviors. The phase refers to a full vertex-to-vertex communication along the input graph [40], i.e., from the `scatter` to `gather` operator.

memory reads while push-based fusion ends with global writes. Thus, they both require a global synchronization between two successive phases, making their fusion infeasible. In contrast, we propose a hybrid fusion template, which uses pull and push graph traversal at the same time to avoid this global synchronization, making dual-phase fusion feasible.

The middle part of Fig. 9 shows the example for the hybrid traversal-based fusion strategy. The outer layer traverses all vertices in the entire graph. The inner layer corresponding to each central vertex consists of two stages: the pull stage and the push stage. In the pull stage, the central vertex serves as the destination vertex, traversing all its connected incoming edges. This mirrors the vertex-to-vertex data flow visible in the pull-based fusion template. Subsequently, in the push stage, the central vertex takes on the role of the source vertex, traversing all its outgoing edges. An optional `ApplyVertex` step can be included between the pull and push stages.

The right part of Fig. 9 details the memory access behaviors of each step in our hybrid fusion strategy, with read-/write address space annotated in the bottom table. The pull stage starts with a `scatter` operator with global space reads, and the push stage ends with a `gather` operator with global space writes. This suggests that our hybrid fusion template can support the fusion chain with at most two phases. We acknowledge the possibility of designing a longer fusion chain by incorporating global memory as an intermediary data buffer and global synchronization [74]. We choose not to explore this option as the additional operations and redundant computations [11] may detriment the overall performance.

5.2 Generating Fusion Candidates

We employ the above fusion templates to generate a comprehensive fusion strategy space by first identifying potential fusion chains and subsequently pruning the fusion candidates through a legality check process, as detailed in Algo. 2.

The identification step involves building matching rules based on our pre-established fusion templates. These rules can cater to either a complete or partial fusion template. Utilizing a BFS-based algorithm, similar to step 1 in Algo. 1,

Algorithm 2: Fusion candidates generation and pruning.

Input: Original graph-centric GNN model $\mathcal{G}_{in}$, Fusion templates $\mathcal{P}$
Output: Fusion candidates $\mathcal{D}$

1 $\mathcal{D} = newSet()$; $\mathcal{H} = newQueue()$; $H.enque(\mathcal{G}_{in})$
2 **while** $\mathcal{H} \neq \{\}$ **do**
3 $\quad \mathcal{G} = \mathcal{H}.dequeue()$;
4 $\quad$ **for** *each* p *in* $\mathcal{P}$ **do**
5 $\quad\quad$ **if** $PatternMatchCheck(\mathcal{G}, p)$ **then**
6 $\quad\quad\quad \mathcal{G}' \leftarrow \text{Fuse}(\mathcal{G}, p)$;
7 $\quad\quad\quad$ **if** $LegalityCheck(\mathcal{G}')$ **then**
8 $\quad\quad\quad\quad \mathcal{H}.enqueue(\mathcal{G}')$;
9 $\quad\quad\quad\quad \mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{G}'$;
10 **return** $\mathcal{D}$;

we generate all potential candidates. The input for pattern matching at this stage is the unfused operator chain, and the output is the fused operator candidates.

We use a `PatternMatchCheck` pass to verify model compatibility with fusion templates, as outlined in Algo. 2, line 5. This check ensures the basic structural requirements are met before proceeding, facilitating the subsequent fusion process and legality assessments. To ensure fusion correctness, the candidate generation stage includes a candidate pruning pass using the `LegalityCheck` function (line 7 of Algo. 2), which evaluates each fusion strategy based on two key rules. The first rule addresses potential data dependency issues, ensuring intermediate data within the fusion candidate is not used by other operators outside its scope. The second rule considers resource constraints, eliminating candidates that require more shared memory than the GPU hardware can provide. By adhering to these rules, we retain viable fusion candidates for the subsequent stage.

5.3 Code Generation and Optimization

Voyager generates high-performance fused GPU kernels through a tile-level tuning and fusion-oriented rewriting

Algorithm 3: Fused Kernel Generation Process

Input: Operator sequence $\mathcal{O} = \{O_1, O_2, \ldots, O_n\}$, hardware platform $\mathcal{H}$
Output: Fused kernel K_{fused} with shared-tile configuration τ

```
// Step 1: Standalone kernel generation with shared tile configuration
```
1 $\tau \leftarrow$ SharedTileTuning($\mathcal{H}, \mathcal{O}$)
2 **for** $O_i \in \mathcal{O}$ **do**
3 | $K_i \leftarrow$ GenerateStandaloneKernel(O_i, τ)
```
// Step 2: Kernel fusion and rewriting
```
4 **for** *each adjacent operator pair* (O_A, O_B) *in* $\mathcal{O}$ **do**
| `// L, C, S: Load, Compute, Store`
5 | $(L_A, C_A, S_A) \leftarrow$ Disassemble(K_A)
6 | $(L_B, C_B, S_B) \leftarrow$ Disassemble(K_B)
7 | **if** *Data dependency* $D_{A \to B}$ *exists* **then**
8 | | Remove S_A, L_B, link $C_A \to C_B$ via shared memory
```
// Step 3: Shared memory pool allocation
```
9 $S \leftarrow$ AllocateSharedMemoryPool($\mathcal{O}$)
10 **for** $O_i \in \mathcal{O}$ **do**
11 | $S_i \leftarrow$ AssignSharedRegion(O_i, S) Attach S_i to K_i
```
// Step 4: Construct final fused kernel
```
12 $K_{fused} \leftarrow$ AssembleFusedKernel($\mathcal{O}, \{K_i\}, \{S_i\}$)
13 **return** K_{fused} with shared-tile configuration τ

process, as shown in Algo. 3. A tile represents the computational data size assigned to a CUDA thread block [48], serving as the fundamental unit for tuning and code generation. This tile-level granularity ensures efficient fusion: all operators within a fusion chain adopt a unified tile configuration, allowing outputs to align seamlessly with inputs, minimizing redundant memory transfers, and enabling flexible adaptation to varying graph sizes without recompilation.

In the first step, we generate standalone kernels for individual operators using existing compiler infrastructures. Neural network operators (NN-OPs) are generated using TVM [5], while graph operators (GOPs) are generated using an enhanced version of GraphIt [3]. GraphIt decouples computation from scheduling, providing the flexibility to define execution strategies independently. Leveraging this, we extend GraphIt's scheduling templates to support feature-dimension parallelism, enabling each thread to process multiple features concurrently, and enhance its scheduling to enable hybrid graph traversal, allowing pull-based and push-based operators to be fused within a single kernel. Both TVM and GraphIt support tile-level tuning. Building on this capability, we introduce `SharedTileTuning` (Line 1 in Algo. 3), a tuning process that determines a globally optimized tile configuration across all fused operators, ensuring consistent tile boundaries and memory alignment.

Next, the fusion-oriented source-to-source rewriter restructures these standalone kernels into a single fused kernel (Step 2 in Algo. 3). Each kernel is first disassembled into

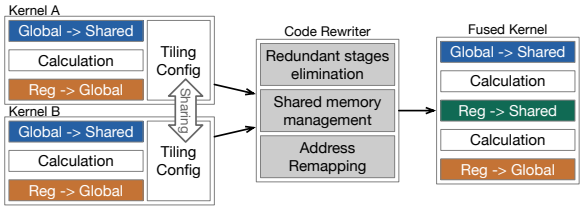


Figure 10. Illustration of code generation process for fused kernel via shared tile configuration tuning and code rewriter.

three canonical stages: data loading (L) from global memory, computation (C), and data storing (S) back to global memory. Stage identification is achieved by applying pattern matching to the structured patterns produced by TVM and GraphIt. Both compilers generate kernels following a well-defined load-compute-store template, where memory accesses and computations appear in distinct and predictable regions. Leveraging this regularity, the rewriter extracts stage boundaries without requiring complex semantic analysis.

After disassembly, the rewriter removes redundant memory stages by analyzing data dependencies (Lines 7-9 in Algo. 3). As shown in Fig. 10, when one operator's output directly feeds into the next, the rewriter bypasses global memory and links them via shared memory, reducing memory traffic while ensuring correctness.

To further support this optimized dataflow, the rewriter performs unified shared memory pool allocation and per-operator address mapping (Step 3 in Algo. 3). It first invokes `AllocateSharedMemoryPool` (Line 10) to compute the total size required for the pool. This calculation considers the peak memory demand across all operators, taking advantage of temporal reuse: if two operators' lifetimes do not overlap, they are allowed to share the same memory space. In the second phase, `AssignSharedRegion` (Line 12) remaps each operator to its own address within the pool, ensuring conflict-free accesses when lifetimes overlap. This lifetime-aware allocation maximizes on-chip memory efficiency without compromising correctness.

Finally, the fused kernel is assembled through `AssembleFusedKernel` procedure (Line 14 in Algo. 3). The fused kernel supports dynamic input graphs by retaining the number of thread blocks as a runtime parameter, allowing the kernel to adapt to the actual graph size without recompilation. This runtime adjustment strictly follows the tile granularity defined by the unified tile configuration, ensuring consistent data partitioning across input graphs of different sizes.

Although our current implementation targets NVIDIA GPUs and leverages CUDA-specific features such as thread blocks and shared memory, the design of *Voyager* is hardware-agnostic. Retargeting the code generator to support different programming models or parallel execution semantics is feasible, as the backend is modular and decoupled from the transformation logic. We leave support for other hardware platforms as future work.

6 Runtime Scheduling

While offline pruning significantly reduces the number of candidates, the total can still vary depending on the complexity of the GNN model. For example, for GIN model, the number of potential candidates can exceed 10^3. To address this challenge, we propose a two-phase runtime scheduler that efficiently optimizes candidate selection from the vast optimization space.

The first phase uses a FLOPS-based white-box cost model to prune reordering candidates, once the input graph and input tensor are available. With the tensor shapes fully defined, the model calculates the computational cost of each candidate with precision, ensuring the selection of the most efficient strategy. By leveraging known shape information, this phase minimizes runtime overhead while narrowing down the set of candidates for the next phase.

The second phase addresses the more complex task of fusion strategy selection, where the optimal strategy depends heavily on the graph structure. For example, push-based fusion is more efficient for src-major-balanced graphs, while pull-based fusion performs better for dst-major-balanced graphs, reducing overhead by avoiding atomic operations. Additionally, the hybrid fusion strategy further minimizes memory access overhead by extending the fusion chain. Fusion strategy selection involves evaluating complex trade-offs between graph traversal efficiency and hardware execution behavior, including memory access patterns and the potential for imbalances in CUDA kernels. Unlike operator reordering, where FLOPS can serve as a straightforward guide for decision-making, these trade-offs are more challenging to quantify using deterministic models alone.

To address this complexity, we employ a black-box predictive model, which dynamically selects the most suitable strategy based on the input graph's characteristics. We construct a synthetic graph database of 128 graphs, chosen to ensure topological diversity, with no overlap between the training and evaluation datasets. The performance tuning data from these graphs serve as the training set for our predictive model, built using the LightGBM framework [26]. The model inputs include graph information listed in Tbl. 3, such as the number of vertices/edges, and maximum and standard deviation of destination/source vertices degree, as well as the input feature size and output class size. During runtime, this information is extracted from the inputs, fed into the prediction model, and used to select the final candidate.

In summary, our two-phase scheduler effectively balances algebraic transformation optimizations with minimal runtime overhead, ensuring practical applicability across diverse GNN workloads. The effectiveness and overhead analysis of the runtime scheduler, along with the offline preprocessing cost, are further evaluated in Sec. 8.3.

7 Methodology

Hardware Setup. To evaluate *Voyager*, we use two different GPUs as our hardware platforms: Tesla V100 [48] and Ampere A100 [47]. Tbl. 5 details our experimental setup.

Target Scenario. We evaluate *Voyager* for full-graph inference on a single GPU, where the working set size corresponds to the number of vertices in the input graph. For large graphs that exceed single-GPU memory capacity, techniques such as graph sampling [69] can partition the graph into smaller subgraphs, which can then be processed independently on each GPU.

Baselines. We choose three start-of-the-art baseline implementations for comparison: 1) Pytorch-Geometric (PyG) [12] is a general GNN framework that is built upon PyTorch. 2) Graphiler [65] is a compiler that optimizes the performance of GNNs on the DGL using static operator reordering and fusion templates. 3) Seastar [63] is a GNN framework with a vertex-centric interface, which supports automatic pull-based operator fusion and code generation. 4) GNNAdvisor (GNNA) [62] presents a runtime system with handwritten CUDA kernels designed to accelerate GNNs on GPUs.

GNN Models. We choose the five representative GNN models that have been widely used by previous works: GCN [30], GIN [66], SageMean [21], GAT [59], PNA [6]. For those models, we use their default configurations of layer, head, and hidden feature parameters, as specified in the official DGL examples, for both our and baseline implementations.

It should be noted that GNNAdvisor [62] only supports GCN and GIN. Seastar [63] does not support the PNA. Additionally, Graphiler [65] lacks support for PNA and multi-head version GAT model. As Graphiler is built upon DGL, we use DGL's built-in functionalities to obtain the results for the GAT and PNA models when Graphiler's support is absent. For other experimental results, we exclude any analysis related to the unsupported models in each baseline.

Datasets. We use 14 datasets that have also been used in previous GNN-related works [27, 32, 55]. Tbl. 3 provides detailed information for these datasets. The datasets vary widely in size, sparsity, and edge distribution, with vertex counts ranging from 2K to 4.21M and edge counts from 5K to 123M. They also exhibit diverse degree distributions, from balanced to highly skewed, along with differences in feature dimensions and class counts. This diversity ensures a comprehensive evaluation across real-world GNN inputs.

Table 5. The experimental setup details.

GPU	NVIDIA Tesla V100 (80 SMs)	Nvidia Ampere A100 (108 SMs)
CPU	Intel(R) Xeon(R) Silver 4210 CPU @ 2.20GHz	Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz
OS	Ubuntu 18.04.5 (kernel 5.4.0)	Ubuntu 20.04.2 (kernel 5.11.0)
Software	GPU Driver Version: 470.57; CUDA Version: 11.6; Pytorch Version: 1.8	

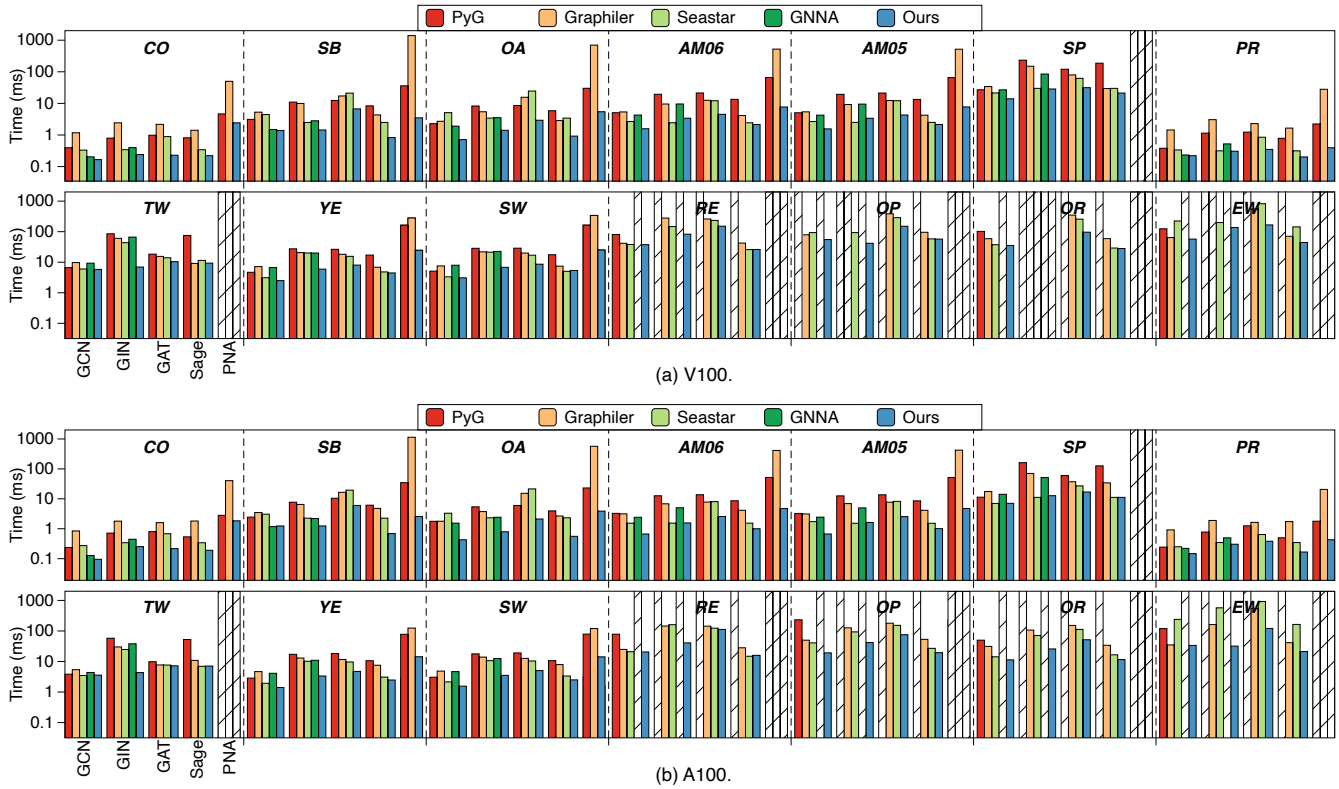


Figure 11. End-to-end time results on two GPUs. Bars with diagonal stripes indicate out-of-memory (OOM) errors.

8 Evaluation

In this section, we aim to evaluate the following points:

- Does the end-to-end performance of *Voyager* outperform existing GNN compilers and manual optimization works?
- How much does each optimization stage in *Voyager* contribute to the overall performance improvement?
- What is the effectiveness of the choices made by *Voyager*, and what is the cost associated with those choices?

8.1 End-to-end Performance Comparisons

Fig. 11 shows the end-to-end execution time on two GPUs. *Voyager* manifests a considerable performance improvement compared to existing state-of-the-art baselines. In particular, the geometric means of the speedup values that *Voyager* provides over PyG, Graphiler, Seastar, and GNNAdvisor are 4.28×, 5.57×, 2.16×, and 2.34× on V100, and 4.63×, 5.62×, 2.35×, and 2.54× on A100, respectively. Additionally, the evaluation includes several large graphs that reach the single-GPU memory limit, leading to OOM failures in some cases and providing a rigorous assessment of scalability.

We observe that *Voyager* achieves different speedup values for different models. Fig. 12 shows per-model speedups, which are highly model-dependent. This can be attributed to the unique structural characteristics within each model, coupled with the disparate execution modes that different baselines employ. We observe that *Voyager* generally achieves

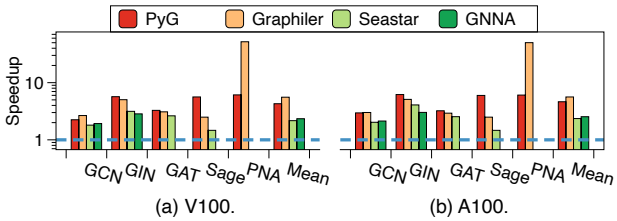


Figure 12. Per-model speedup averaged on all datasets. The speedup values are calculated over different baselines.

more speedups for complex GNN models like GIN and PNA. The reason is that complex GNN models expose more opportunities for our algebraic transformation including the long-range reorder and traversal-aware fusion. We also achieve the best speedup for the PNA model [6] over the Graphiler [65] baseline. The reason is that this DGL-based compiler adopts a strategy that groups vertices of the same degree into one bucket and performs independent calculations for each bucket, leading to significant performance loss.

Fig. 13 shows per-dataset speedups. Compared to existing works, we have different speedup ratios for different datasets. This variance can be attributed to the substantial differences in structural features present across diverse graph topologies. In particular, both small graphs such as CO and large graphs such as EW exhibit comparable speedup, demonstrating *Voyager*'s consistent effectiveness across different graph

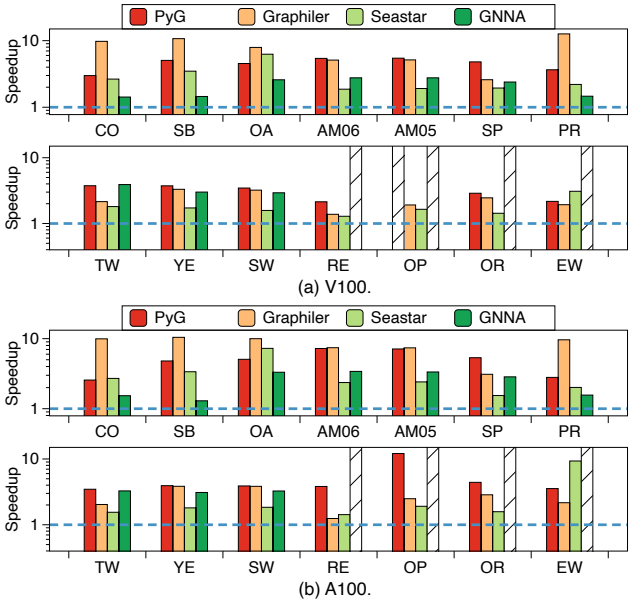


Figure 13. Per-dataset speedup averaged on all models. Bars with diagonal stripes indicate out-of-memory (OOM) errors on all models.

scales. This confirms that our optimizations are not biased toward any specific size range. On average, it is observed that directed graphs yield a higher performance improvement relative to undirected graphs. This performance gain is primarily due to the expanded strategy space facilitated by push fusion, which demonstrates superior results for directed graphs. Furthermore, this outcome underlines the limitations of current baselines, which can only deliver high performance for a restricted subset of datasets owing to their lack of adaptive strategy selection.

We also evaluate *Voyager*'s optimization capabilities in comparison with the baseline systems. For example, *Voyager* explores reordering opportunities, applying transformation rules that significantly enhance execution efficiency. *Voyager* dynamically selects applicable rules based on workload characteristics. For instance, Sage benefits primarily from rules 1, 2, and 3 of Tbl. 4, while GIN leverages rules 1, 3, 6, and 8. GCN mainly applies rules 1 and 3, whereas GAT utilizes rules 3 and 5. Although DGL applies optimizations in a manual and limited manner, no baseline system has yet implemented

Table 6. Comparison of computation overhead (#FLOPS) for graph operators between GNNAdvisor and *Voyager* in GIN.

Dataset	CO	SB	OA	AM06	AM05	SP	PR
Baseline	8.1e6	4.0e8	2.2e8	5.4e8	5.4e8	4.7e9	1.5e7
Ours	3.9e5	1.7e8	9.3e7	2.7e8	2.7e8	9.6e8	1.1e7
Gain	21.1×	2.4×	2.4×	2.0×	2.0×	4.8×	1.4×

Dataset	TW	YE	SW	RE	OP	OR	EW
Baseline	2.0e9	5.0e8	5.1e8	7.6e10	2.0e10	2.2e10	3.7e10
Ours	9.5e7	2.4e8	2.6e8	9.2e9	9.9e9	7.7e9	7.7e9
Gain	21.0×	2.1×	2.0×	8.3×	2.1×	2.9×	4.8×

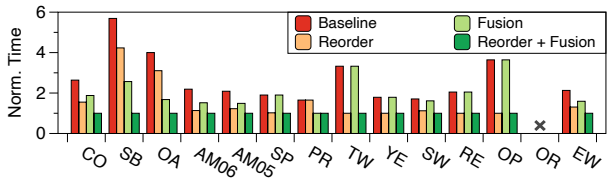


Figure 14. Performance comparison between different *Voyager* versions for GIN on V100. OR encounter out-of-memory errors on all versions.

automated optimization for the complex demands of models like GIN. PNA, due to its large model size and complex computation patterns, provides abundant fusion opportunities, further amplifying *Voyager*'s performance gains. This adaptability enables *Voyager* to optimize a diverse range of GNN models beyond manual tuning efforts. This underscores the significant benefits of *Voyager*'s automated techniques.

In Tbl. 6, we present a detailed comparison of the computational overhead over the baseline GNNAdvisor [62] in terms of #FLOPS metric for graph operators in the GIN model across different input graphs. The findings reveal that *Voyager* significantly reduces computational overhead, achieving up to a 20× reduction over the baseline, which is one of the major improvement sources.

8.2 Optimization Strategy Space Analysis

Voyager incorporates multiple algebraic transformation optimization passes, including operator reordering (Sec. 4) and operator fusion (Sec. 5). These passes sculpt an adequately large optimization space, thereby promoting the overall enhancement of computational performance. To study the impact of these optimization passes on performance, we construct the following baselines and conduct meticulous experiments to quantitatively ascertain their effectiveness.

We set up four different versions for our experiments: the `baseline`, `reorder`, `fusion`, and `reorder+fusion`. The `baseline` version does not include any optimization passes, relying on a default model and utilizing the pull-based operator fusion method. Meanwhile, `reorder` version incorporates long-range operator reordering, and `fusion` version applies dynamic operator fusion on top of `baseline`. Lastly, `reorder+fusion` applies all optimization passes. In our experiments designed to evaluate the efficacy of the strategy space, the performance result is determined through a grid search for the versions that utilize multiple strategies.

We use the GIN model [66] in this experiment and collect the performance results in Fig. 14. Due to the space limit, we omit the results for the other models. The GIN results show that applying optimization passes can significantly improve performance, with the greatest gains achieved by using all optimization passes in the `reorder+fusion` version. Furthermore, the `reorder` version of `CO` and `TW` shows a significant performance improvement compared to `baseline`, but `PR` does not show much improvement. This is because the input

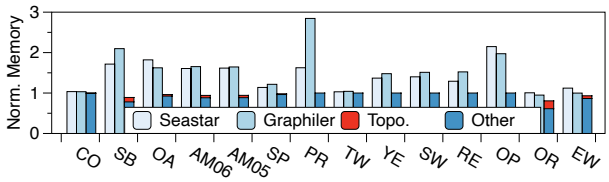


Figure 15. Memory comparison of various works for Sage on V100. 'Topo.' and 'Other' represent additional topology and residual memory requirements in *Voyager*, respectively.

feature sizes for CO and TW are larger (Tbl. 3), for which the performance gains from computation graph rewriting are more significant. Additionally, datasets featuring a balanced pattern, such as SB and OA, yielded more significant performance enhancements through the use of dynamic operator fusion in the `fusion` version.

The varying degrees of performance improvement across different datasets underscore the importance of applying a comprehensive set of optimization strategies, as enabled by our system. It also highlights the necessity of the input-adaptive selection process that we evaluate individually next.

8.3 Overhead and Prediction Accuracy Studies

Memory Overhead Analysis. Compared to existing baselines, *Voyager* concurrently employs pull- and push-based methods, necessitating both destination- and source-major topologies. For undirected graphs, these topologies are identical, avoiding extra storage. For directed graphs, while this could introduce additional storage overhead, it remains minimal compared to overall memory usage. Moreover, *Voyager*'s fusion strategy effectively reduces intermediate data storage, mitigating this impact. To analyze memory overhead, we measure the memory usage of the Sage model [21] on a V100 GPU and compare *Voyager* with Seastar [63] and Graphiler [65]. As shown in Fig. 11, these two baselines exhibit the fewest out-of-memory failures, making them the most representative for evaluating memory efficiency. As illustrated in Fig. 15, *Voyager* exhibits a consistently lower memory footprint overall. On average, the overhead accounts for only 3.8% of total memory usage, with minimal impact on overall efficiency. These results highlight both the efficiency and scalability of *Voyager* across diverse graph structures.

Offline Overhead Analysis. *Voyager*'s offline preprocessing consists of four stages: candidate generation, kernel generation, candidate profiling, and model training, with their overhead breakdown shown in Fig. 16. The first two stages form candidate preparation, where algebraic transformation opportunities are identified and optimized GPU kernels are generated. Our pruning strategy significantly reduces the cost of candidate generation, which accounts for 2% of the total preprocessing time. Meanwhile, tile-level profiling, restricted to a single CTA, minimizes kernel generation overhead, contributing only 3% to the overall cost.

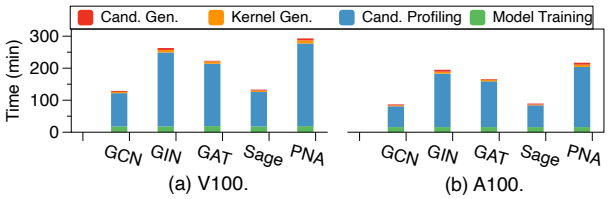


Figure 16. Offline preprocessing cost breakdown on V100 and A100 for different GNN models.

The latter two stages constitute model preparation, where execution data is collected and used to train a predictive model. Candidate profiling dominates the offline cost, comprising over 80% of the total preprocessing time, while model training remains lightweight at less than 15%.

These offline costs are amortized across multiple inference tasks, making the one-time investment negligible relative to runtime performance gains. Compared to GNNAdvisor, which requires manual tuning, *Voyager* eliminates the need for extensive human intervention. Additionally, unlike DNN compilers such as Ansor [73], which require days of tuning to support dynamic-shape workloads [72], *Voyager* achieves comparable optimizations within a few hours, making its overhead practical for real-world usage. While these preprocessing costs are already manageable, further optimizations remain possible. Future improvements, such as reusing trained models across different GNN workloads, can further reduce preprocessing overhead and enhance adaptability.

Runtime Overhead Analysis. The runtime overhead consists of three components: the white-box cost model, graph feature extraction, and inference time of the black-box prediction. We compare *Voyager* with GNNAdvisor, as it implements manually optimized pull-based fusion, making it the existing SOTA baseline. Fig. 17 illustrates *Voyager*'s overhead breakdown on GIN and compares its performance with GNNAdvisor. The runtime overhead averages just 6.8%, which is marginal. Despite this overhead, *Voyager* matches GNNAdvisor in execution efficiency while avoiding OOM errors. This advantage stems from its automated operator reordering and hybrid fusion, improving performance and optimizing memory usage beyond manual pull-based fusion.

Prediction Accuracy. Our compiler adopts an adaptive selection module to select the performance-optimal candidate. To quantify its efficacy, we employ a grid search to

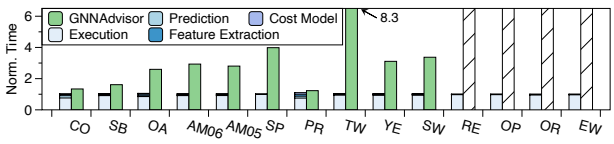


Figure 17. Execution time breakdown of *Voyager* for GIN on A100, benchmarking against GNNAdvisor. Bars with diagonal stripes indicate OOM errors on GNNAdvisor.

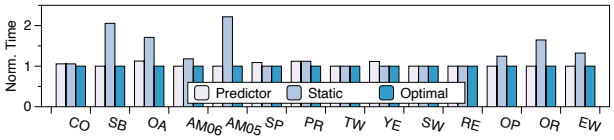


Figure 18. Performance comparison between predictor, static strategy, and grid search for GCN on V100.

find the best candidate for ***each*** dataset when running the GCN model [30] on the V100 GPU, labeled as 'Optimal'. We also identify a single static strategy, labeled as 'Static', which achieves the highest overall average performance when applied across all datasets. This strategy is selected once and applied uniformly, without considering any per-graph characteristics. Fig. 18 reveals that our predictor consistently yielded performance metrics comparable to the optimal performance through brute-force grid search. Furthermore, our predictor outperforms the static strategy by an average of 18.7% (up to 2×), validating our input-adaptive methods.

9 Related Work

Traditional Graph Processing Systems. Numerous graph processing systems [15, 28, 29, 31, 46, 53, 61] have been proposed to accelerate traditional graph algorithm processing. Many of these research works have tried to explore different traversal strategies, typically selecting strategies at the inter-kernel level based on graph sparsity [61] or connectivity patterns [71]. GNNs require coordinated graph operations and dense computation. *Voyager* focuses on intra-kernel optimization, fusing graph and neural network operators while selectively applying pull and push strategies within fused kernels. Unlike traditional graph systems that optimize traversal alone, *Voyager* co-optimizes data movement and execution, balancing workloads and reducing redundant memory access.

GNN Frameworks and Compilers. DGL [60] and PyG [12] are two popular GNN frameworks built on DNN frameworks such as PyTorch [49]. Roc [24], NeuGraph [40], and AliGraph [78] target large-scale distributed GNN workloads. Seastar [63] employs a vertex-centric programming model with graph tags to trace computation graphs, enabling automatic operator fusion and code generation. Graphiler [65], designed for DGL, optimizes performance through static operator reordering and fusion. However, these solutions are constrained by a limited optimization scope and overlook the diversity of inputs. uGrapher [75] offers input-adaptive optimization at the kernel level through a unified interface, but misses out on the broader optimization opportunities provided by algebraic transformations. Distinctively, *Voyager* is the first work to leverage an input-adaptive approach for algebraic transformation, facilitating more comprehensive and efficient GNN optimizations.

GPU-Based GNN Optimization. Many studies have focused on optimizing GPUs at the architecture [18–20, 22], scheduling [7, 8, 17, 36], and algorithm [16, 37, 76, 77] levels, inspiring system-level optimization strategies for GNN execution. Ge-SpMM [23] introduces manually-crafted kernels for `SpMM`-like operations, while GNNAdvisor [62] offers a runtime system with tailored implementations for GCN and GIN enhancement. Other efforts, such as dgNN [70], focus on manual optimization strategies for GNN computation graphs, and FusedMM [51] presents a fused `SpMM` and `SDDMM` kernel for high-performance GNN computations. However, such manual techniques lack the analysis and modeling process, which constrains their use scenarios in general GNN models with growing complexity.

Algebraic Transformation for DNNs. Many existing works attempt to address the algebraic transformation optimizations of DNNs. JAX [2], TASO [25], TenSAT [67], TorchDynamo [50], LazyTensor [57] and XLA [54] exploit operator reordering optimizations. DNNFusion [45], Rammer [39] HFuse [34], and AStitch [74] explore fusion opportunities. However, GNNs pose unique challenges owing to the diverse input graphs and associated graph operators that traverse and compute along the input graph. Existing DNN compilers overlook the unique input graph characteristics, which limit their optimization scopes and efficiencies.

10 Conclusion

In this work, we propose *Voyager*, an end-to-end compiler designed to enable graph-centric and input-adaptive algebraic transformations for GNNs. *Voyager* establish a comprehensive optimization framework that facilitates operator reordering and fusion while integrating input attributes into adaptive runtime strategy selection. This innovative approach yields an average 3.5× speedup over GNN frameworks, compilers, and manual optimizations.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (NSFC) Grants (U21B2017 and 62222210), the Shanghai Qi Zhi Institute Innovation Program SQZ202316, and the Provincial Key Research and Development Program of Hubei (No. 2023BAB065). This work was also supported by Alibaba Innovative Research Program, and Alibaba Research Intern Program. We would like to thank the anonymous reviewers for their constructive feedback and comments to improve this work. We also thank our shepherd, Nishil Talati, for his support and guidance in preparing this paper for publication. We also thank Qing Wang, Hao Lv for beneficial discussion and useful comments. Any opinions, findings, and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

References

[1] Alaa Bessadok, Mohamed Ali Mahjoub, and Islem Rekik. Graph neural networks in network neuroscience. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. Jax: composable transformations of python+ numpy programs. *Version 0.2*, 5:14–24, 2018.

[3] Ajay Brahmakshatriya, Yunming Zhang, Changwan Hong, Shoaib Kamil, Julian Shun, and Saman Amarasinghe. Compiling graph applications for gpu s with graphit. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 248–261. IEEE, 2021.

[4] Ines Chami, Sami Abu-El-Haija, Bryan Perozzi, Christopher Ré, and Kevin P. Murphy. Machine learning on graphs: A model and comprehensive taxonomy. *ArXiv*, abs/2005.03675, 2020.

[5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.

[6] Gabriele Corso, Luca Cavalleri, Dominique Beaini, Pietro Liò, and Petar Veličković. Principal neighbourhood aggregation for graph nets. *Advances in Neural Information Processing Systems*, 33:13260–13271, 2020.

[7] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. {DVABatch}: Diversity-aware {Multi-Entry}{Multi-Exit} batching for efficient processing of {DNN} services on {GPUs}. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 183–198, 2022.

[8] Weihao Cui, Han Zhao, Quan Chen, Ningxin Zheng, Jingwen Leng, Jieru Zhao, Zhuo Song, Tao Ma, Yong Yang, Chao Li, et al. Enable simultaneous dnn services based on deterministic operator overlap and precise latency prediction. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.

[9] Yingtong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 315–324, 2020.

[10] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. Graph neural networks for social recommendation. In *The World Wide Web Conference*, pages 417–426. ACM, 2019.

[11] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. Graph random neural networks for semi-supervised learning on graphs. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[12] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

[13] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, et al. Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 922–936. IEEE, 2020.

[14] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. PMLR, 2017.

[15] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. {PowerGraph}: Distributed {Graph-Parallel} computation on natural graphs. In *10th USENIX symposium on operating systems design and implementation (OSDI 12)*, pages 17–30, 2012.

[16] Cong Guo, Yuxian Qiu, Jingwen Leng, Xiaotian Gao, Chen Zhang, Yunxin Liu, Fan Yang, Yuhao Zhu, and Minyi Guo. SQuant: On-the-fly data-free quantization via diagonal hessian approximation. In *International Conference on Learning Representations*, 2022.

[17] Cong Guo, Yuxian Qiu, Jingwen Leng, Chen Zhang, Ying Cao, Quanlu Zhang, Yunxin Liu, Fan Yang, and Minyi Guo. Nesting forward automatic differentiation for memory-efficient deep neural network training. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*, pages 738–745. IEEE, 2022.

[18] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[19] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1414–1433. IEEE, 2022.

[20] Cong Guo, Yangjie Zhou, Jingwen Leng, Yuhao Zhu, Zidong Du, Quan Chen, Chao Li, Bin Yao, and Minyi Guo. Balancing efficiency and flexibility for dnn acceleration via temporal gpu-systolic array integration. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

[21] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[22] Weiming Hu, Haoyan Zhang, Cong Guo, Yu Feng, Renyang Guan, Zhendong Hua, Zihan Liu, Yue Guan, Minyi Guo, and Jingwen Leng. M-ant: Efficient low-bit group quantization for llms via mathematically adaptive numerical type. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1112–1126. IEEE, 2025.

[23] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. Gespmm: General-purpose sparse matrix-matrix multiplication on gpus for graph neural networks. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. IEEE, 2020.

[24] Zhihao Jia, Sina Lin, Mingyu Gao, Matei Zaharia, and Alex Aiken. Improving the accuracy, scalability, and performance of graph neural networks with roc. *Proceedings of Machine Learning and Systems*, 2:187–198, 2020.

[25] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.

[26] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

[27] Kristian Kersting, Nils M. Kriege, Christopher Morris, Petra Mutzel, and Marion Neumann. Benchmark data sets for graph kernels, 2016.

[28] Zuhair Khayyat, Karim Awara, Amani Alonazi, Hani Jamjoom, Dan Williams, and Panos Kalnis. Mizan: a system for dynamic load balancing in large-scale graph processing. In *Proceedings of the 8th ACM European conference on computer systems*, pages 169–182, 2013.

[29] Farzad Khorasani, Keval Vora, Rajiv Gupta, and Laxmi N Bhuyan. Cusha: vertex-centric graph processing on gpus. In *Proceedings of the 23rd international symposium on High-performance parallel and distributed computing*, pages 239–252, 2014.

[30] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[31] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. {GraphChi}:{Large-Scale} graph computation on just a {PC}. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pages 31–46, 2012.

[32] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, June 2014.

[33] Ao Li, Zhou Qin, Runshi Liu, Yiqun Yang, and Dong Li. Spam review detection with graph convolutional networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 2703–2711, 2019.

[34] Ao Li, Bojian Zheng, Gennady Pekhimenko, and Fan Long. Automatic horizontal fusion for gpu kernels. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 14–27. IEEE, 2022.

[35] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):708–727, 2020.

[36] Zihan Liu, Jingwen Leng, Zhihui Zhang, Quan Chen, Chao Li, and Minyi Guo. Veltair: towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 388–401, 2022.

[37] Zihan Liu, Xinhao Luo, Junxian Guo, Wentao Ni, Yangjie Zhou, Yue Guan, Cong Guo, Weihao Cui, Yu Feng, Minyi Guo, et al. Vq-llm: High-performance code generation for vector quantization augmented llm inference. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1496–1509. IEEE, 2025.

[38] Yucheng Low, Joseph Gonzalez, Aapo Kyrola, Danny Bickson, Carlos Guestrin, and Joseph M Hellerstein. Distributed graphlab: A framework for machine learning in the cloud. *arXiv preprint arXiv:1204.6078*, 2012.

[39] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. Rammer: Enabling holistic deep learning compiler optimizations with {rTasks}. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 881–897, 2020.

[40] Lingxiao Ma, Zhi Yang, Youshan Miao, Jilong Xue, Ming Wu, Lidong Zhou, and Yafei Dai. {NeuGraph}: Parallel deep neural network computation on large graphs. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 443–458, 2019.

[41] Kelong Mao, Jieming Zhu, Xi Xiao, Biao Lu, Zhaowei Wang, and Xiuqiang He. Ultragcn: ultra simplification of graph convolutional networks for recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pages 1253–1262, 2021.

[42] Ke Meng, Jiajia Li, Guangming Tan, and Ninghui Sun. A pattern based algorithmic autotuner for graph processing on gpus. In *Proceedings of the 24th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP 2019, Washington, DC, USA, February 16-20, 2019*, pages 201–213. ACM, 2019.

[43] Steven Muchnick et al. *Advanced compiler design implementation*. Morgan kaufmann, 1997.

[44] David F. Nettleton. Data mining of social networks represented as graphs. *Comput. Sci. Rev.*, 7:1–34, 2013.

[45] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 883–898, 2021.

[46] Amir Hossein Nodehi Sabet, Junqiao Qiu, and Zhijia Zhao. Tigr: Transforming irregular graphs for gpu-friendly graph processing. *ACM SIGPLAN Notices*, 53(2):622–636, 2018.

[47] NVIDIA. Nvidia a100 tensor core gpu architecture. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf, 2021.

[48] Tesla NVIDIA. Nvidia tesla v100 gpu architecture, 2017.

[49] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[50] PyTorch Contributors. TorchDynamo. https://pytorch.org/docs/master/dynamo/, 2022.

[51] Md Khaledur Rahman, Majedul Haque Sujon, and Ariful Azad. Fusedmm: A unified sddmm-spmm kernel for graph embedding and graph neural networks. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 256–266. IEEE, 2021.

[52] Ryan Rossi and Nesreen Ahmed. The network data repository with interactive graph analytics and visualization. In *Twenty-ninth AAAI conference on artificial intelligence*, 2015.

[53] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488, 2013.

[54] Amit Sabne. Xla : Compiling machine learning for peak performance, 2020.

[55] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 29(3):93–93, 2008.

[56] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.

[57] Alex Suhan, Davide Libenzi, Ailing Zhang, Parker Schuh, Brennan Saeta, Jie Young Sohn, and Denys Shabalin. Lazytensor: combining eager execution with domain-specific compilers. *arXiv preprint arXiv:2102.13267*, 2021.

[58] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 267–284. USENIX Association, 2022.

[59] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[60] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.

[61] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*, pages 1–12, 2016.

[62] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. {GNNAdvisor}: An adaptive and efficient runtime system for {GNN} acceleration on {GPUs}. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 515–531, 2021.

[63] Yidi Wu, Kaihao Ma, Zhenkun Cai, Tatiana Jin, Boyang Li, Chenguang Zheng, James Cheng, and Fan Yu. Seastar: vertex-centric programming for graph neural networks. In *Proceedings of the Sixteenth European Conference on Computer Systems*, pages 359–375, 2021.

[64] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.

[65] Zhiqiang Xie, Minjie Wang, Zihao Ye, Zheng Zhang, and Rui Fan. Graphiler: Optimizing graph neural networks with message passing data flow graph. *Proceedings of Machine Learning and Systems*, 4:515–528, 2022.

[66] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.

[67] Yichen Yang, Phitchaya Mangpo Phothilimthana, Yisu Remy Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. Equality saturation for tensor graph superoptimization. In Alex Smola, Alex Dimakis, and Ion Stoica, editors, *Proceedings of Machine Learning and Systems 2021, MLSys 2021, virtual, April 5-9, 2021*. mlsys.org, 2021.

[68] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. Wise: Predicting the performance of sparse matrix vector multiplication with machine learning. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 329–341, 2023.

[69] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor K. Prasanna. Graphsaint: Graph sampling based inductive learning method. *ArXiv*, abs/1907.04931, 2020.

[70] Hengrui Zhang, Zhongming Yu, Guohao Dai, Guyue Huang, Yufei Ding, Yuan Xie, and Yu Wang. Understanding gnn computation graph: A coordinated computation, io, and memory perspective. *Proceedings of Machine Learning and Systems*, 4:467–484, 2022.

[71] Yunming Zhang, Mengjiao Yang, Riyadh Baghdadi, Shoaib Kamil, Julian Shun, and Saman Amarasinghe. Graphit: A high-performance graph dsl. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–30, 2018.

[72] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Joshua Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. Dietcode: Automatic optimization for dynamic tensor programs. *Proceedings of Machine Learning and Systems*, 4:848–863, 2022.

[73] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, pages 863–879, 2020.

[74] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, et al. Astitch: enabling a new multi-dimensional optimization space for memory-intensive ml training and inference on modern simt architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 359–373, 2022.

[75] Yangjie Zhou, Jingwen Leng, Yaoxu Song, Shuwen Lu, Mian Wang, Chao Li, Minyi Guo, Wenting Shen, Yong Li, Wei Lin, Xiangwen Liu, and Hanqing Wu. ugrapher: High-performance graph operator computation via unified abstraction for graph neural networks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, pages 878–891. ACM, 2023.

[76] Yangjie Zhou, Yaoxu Song, Jingwen Leng, Zihan Liu, Weihao Cui, Zhendong Zhang, Cong Guo, Quan Chen, Li Li, and Minyi Guo. Adaptgear: Accelerating gnn training via adaptive subgraph-level kernels on gpus. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*, pages 52–62, 2023.

[77] Yangjie Zhou, Mengtian Yang, Cong Guo, Jingwen Leng, Yun Liang, Quan Chen, Minyi Guo, and Yuhao Zhu. Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*, pages 214–225. IEEE, 2021.

[78] Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. Aligraph: a comprehensive graph neural network platform. *arXiv preprint arXiv:1902.08730*, 2019.